

1. Objectif et domaine

Ce projet implemente un systeme Agentic RAG dans le domaine de l'assurance. L'objectif est de repondre a des questions sur les contrats, les sinistres, les exclusions, les procedures de reclamation et certaines contraintes reglementaires. Le systeme s'appuie sur une base documentaire locale et sur une architecture LangGraph construite manuellement, sans utiliser `create_agent` de LangChain.

2. Construction de la base documentaire

La base documentaire est organisee dans le dossier `data/` par categorie : automobile, sante, habitation, procedures de sinistre, FAQ et reglementation. Le script `vectorstore/ingest.py` lit les fichiers `.txt`, `.md` et `.pdf`, extrait le texte, puis le decoupe en chunks de 1200 caracteres avec un chevauchement de 180 caracteres. Chaque chunk est stocke dans ChromaDB avec des metadonnees : source, categorie et numero du chunk.

La vectorisation est configuree dans `tools/retriever.py`. Le projet supporte trois modes d'embedding : `hash` pour les tests locaux sans API, `openai` pour une meilleure qualite semantique, et `sentence_transformers` pour un modele local. ChromaDB est persiste dans `vectorstore/chroma_db`.

3. Modeles LLM et outils

Le module `tools/llm.py` centralise les appels aux modeles. Il supporte un mode `mock` pour les tests, `OpenAI`, `Gemini`, `xAI/Grok` et `Ollama`. Les outils principaux sont :

- `retriever.py` : recherche vectorielle dans ChromaDB avec bonus par categorie.
- `grader.py` : evaluation de la pertinence des documents recuperes.
- `rewrite.py` : reformulation de la question si les documents sont faibles.
- `summarizer.py` : synthese des passages utiles avant generation.
- `evaluator.py` : auto-evaluation de la reponse selon qualite, ancrage documentaire et completude.

Ces outils donnent au systeme un comportement agentique : il ne se limite pas a recuperer des documents, il decide si la preuve est suffisante, reformule au besoin, puis evalue sa propre reponse.

4. Architecture LangGraph, state et memoire

Le graphe est defini dans `graph/graph.py` avec `StateGraph`. Les noeuds sont : `analyze_question`, `retrieve_documents`, `grade_documents`, `rewrite_question`, `summarize_documents`, `generate_answer`, `self_evaluate_answer` et `save_memory`. Une transition

Page 1

conditionnelle apres grade_documents envoie le flux soit vers la synthese, soit vers la reformulation lorsque le score de pertinence est insuffisant.

Le state est decrit dans graph/state.py. Il contient la question, l'analyse, les documents, les scores de pertinence, la synthese, la reponse, l'evaluation, le compteur de reformulation, la memoire et le temps de reponse. La memoire est geree avec MemorySaver dans LangGraph et avec st.session_state dans l'interface Streamlit.

La visualisation du graphe est disponible dans docs/graph.svg et sa version Mermaid dans docs/graph.mmd.

5. Interface et simulation

L'application Streamlit app.py permet de poser une question, d'afficher la reponse et d'ouvrir une section Trace. Cette trace expose l'analyse de la question, la reformulation eventuelle, le score de pertinence, l'evaluation de la reponse, le temps de reponse et les sources recuperees.

Pour la simulation, deux exemples sont recommandes :

- Question simple : "What documents are usually needed to file a motor insurance claim?"
- Question complexe : "If my car was damaged while being used for paid delivery work, how should I determine whether my motor policy covers the claim?"

6. Evaluation

Le script evaluation/evaluate.py teste le systeme sur 10 questions simples et 10 questions complexes. Pour chaque question, il enregistre la reponse, la qualite estimee, le score d'ancrage documentaire, la completude, le temps de reponse, la pertinence moyenne des documents recuperes et le nombre de sources.

Lors d'un test local en mode mock, les moyennes obtenues etaient :

Suite	Questions	Qualite moyenne	Temps moyen	Pertinence documents
---	---	---	---	---
Simple	10	0.70	0.22 s	0.75
Complexe	10	0.70	0.06 s	0.75

Ces resultats valident le fonctionnement technique complet. Pour une evaluation finale, il est preferable de relancer le script avec un vrai LLM et, si possible, des embeddings semantiques (openai ou sentence_transformers) afin d'obtenir une analyse plus fiable.

7. Limites et pistes d'amelioration

La base documentaire actuelle est volontairement courte et sert surtout à démontrer l'architecture. La qualité des réponses dépend fortement du fournisseur LLM et du mode d'embedding choisi. Le mode hash est utile pour les tests, mais il reste moins pertinent qu'un modèle d'embedding sémantique.

Les améliorations possibles sont : enrichir la base documentaire avec des contrats réels anonymisés, ajouter un reranker, conserver une mémoire conversationnelle plus structurée, ajouter des tests unitaires pour les nœuds du graphe, comparer plusieurs modèles LLM, et produire une évaluation humaine des réponses en plus de l'auto-évaluation automatique.

Page 3